Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

# Machine Learning in a Nutshell

## 15-288 Spring '26

## Lecture 3:
## SL workflow, SL Canonical Problem

Teacher:
Gianni A. Di Caro

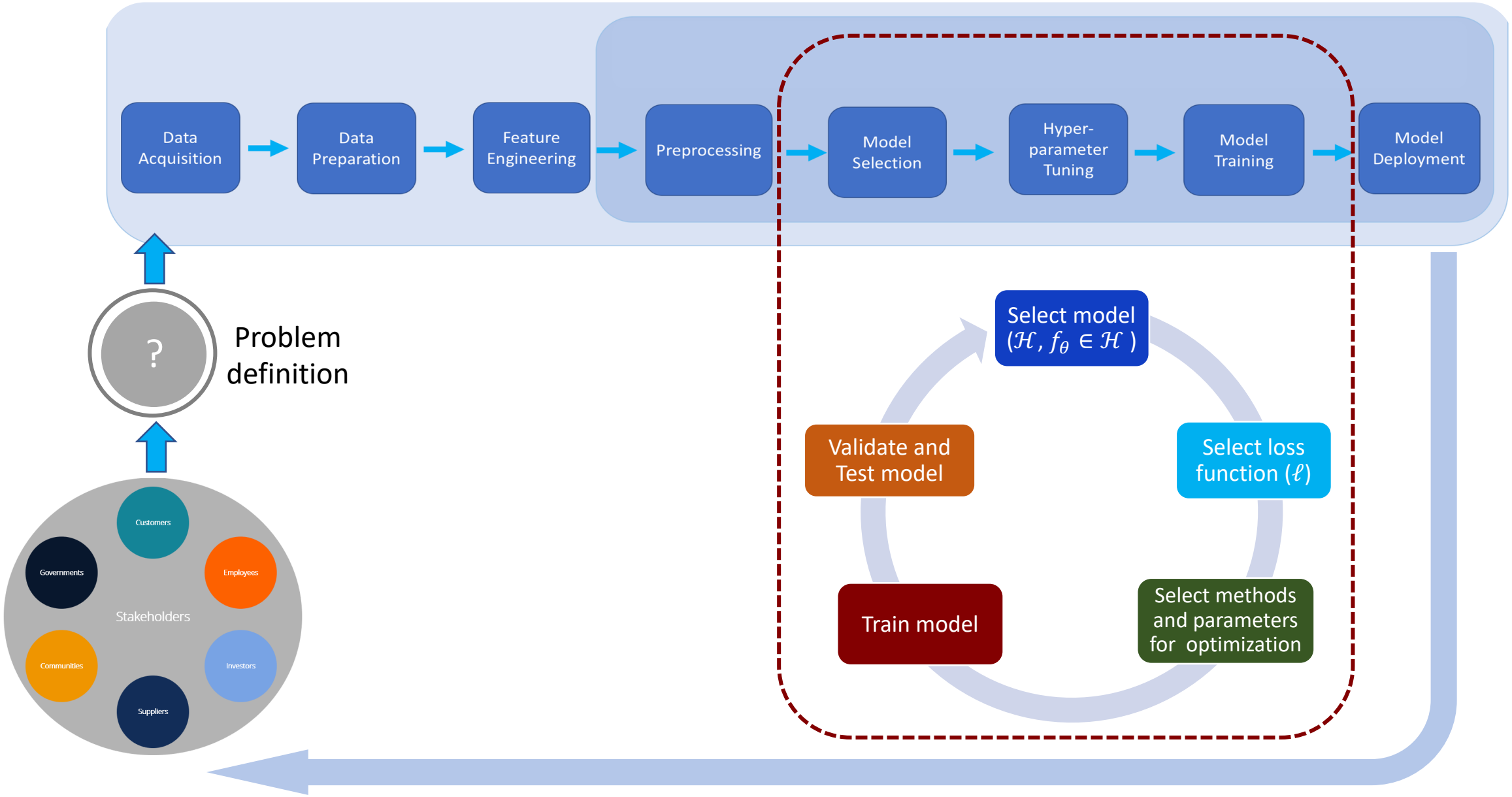
جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

# Warm-up Quiz time

1. I have a dataset of JPEG images featuring cats, where each image has a label telling the weight of the cat.
   a) Can I effectively use the dataset to build a predictor, $f$, of cat weight?
   b) If the answer to (a) is Yes, would this be a classification or a regression task?
   c) Will I run into troubles if somebody will ask me to use $f$ to predict the weight of a cat from a PNG image given that my dataset was made of JPEG images?
   d) Which features would you use for learning in this case?
   e) Let's assume that we don't have labels, but still, we would like to group cats according to their weight. What type of ML task would this be? Do you think would this be an easy task to accomplish?

2. Can you mention at least three reasons why data labeling could be complicate or error-prone?

# Machine Learning Workflow / Lifecycle



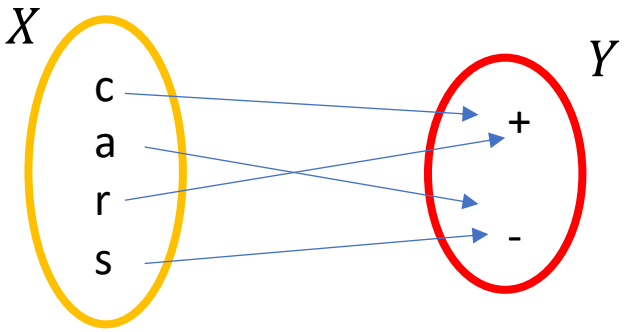
# Supervised Learning: A Classification example workflow

**Data set $X$:** cars

**Labeling:**

$+$ *Family car*
(positive example)

$-$ No family car
(negative example)




**Task:**

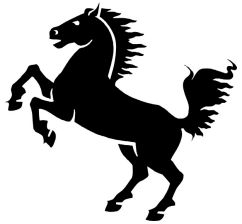
Learn class C $\equiv$ *Family car*

$C(car)$ ? $^{+}/_{-}$

Learn the mapping:

$f: X \rightarrow Y$

**Features?**



Engine power

Price

~~Color~~
~~Size~~
~~Traction~~
Consumption
....

**Performance**: minimize misclassification errors on training set, aiming to correctly predict unseen data

# Dataset, features

- A car is represented as a numeric vector of **two features**:

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \equiv \begin{bmatrix} \$ \\ \text{cc} \end{bmatrix}, \quad \boldsymbol{x} \in \mathbb{R}^2$$

- The **label** of a car is numeric, binary, value that denotes its type:

$$y = \begin{cases} 1 & \text{if } \boldsymbol{x} \text{ is a positive example (family car)} \\ 0 & \text{if } \boldsymbol{x} \text{ is a negative example} \end{cases}$$

Can I use +1, -1 instead?

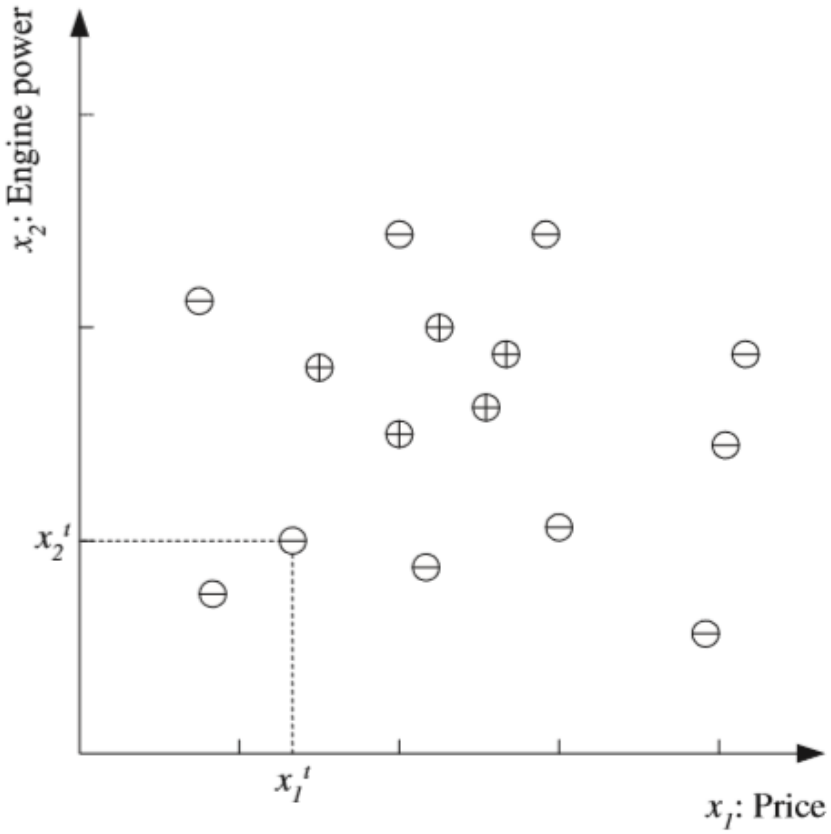
- In the **dataset** $D$, a *car example* $i$ is represented by an ordered pair $(\boldsymbol{x}^{(i)}, y^{(i)})$, and there are $m$ examples in the data set:

$$D = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$$

# Acquire useful information by EDA

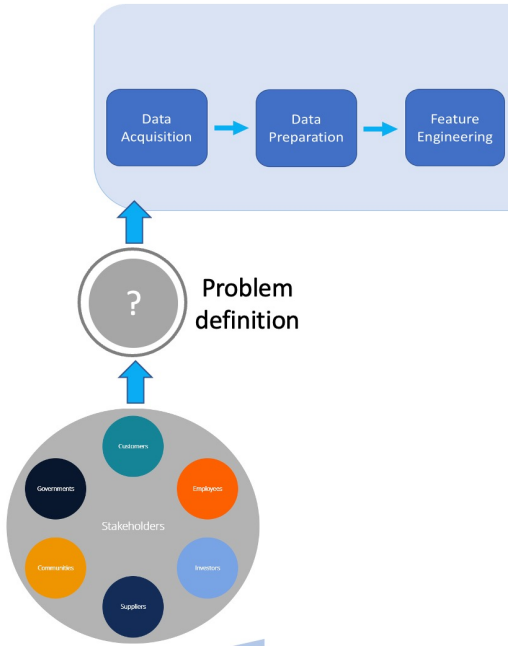
❖ **Exploratory Data Analysis (EDA)**:

- ✓ Visualization of the dataset in the two-dim *feature space*
- ✓ Statistics about the dataset of examples
  - 15 data entries, 5 family cars
  - Range of values of features
  - Median price for + and for −
  - …



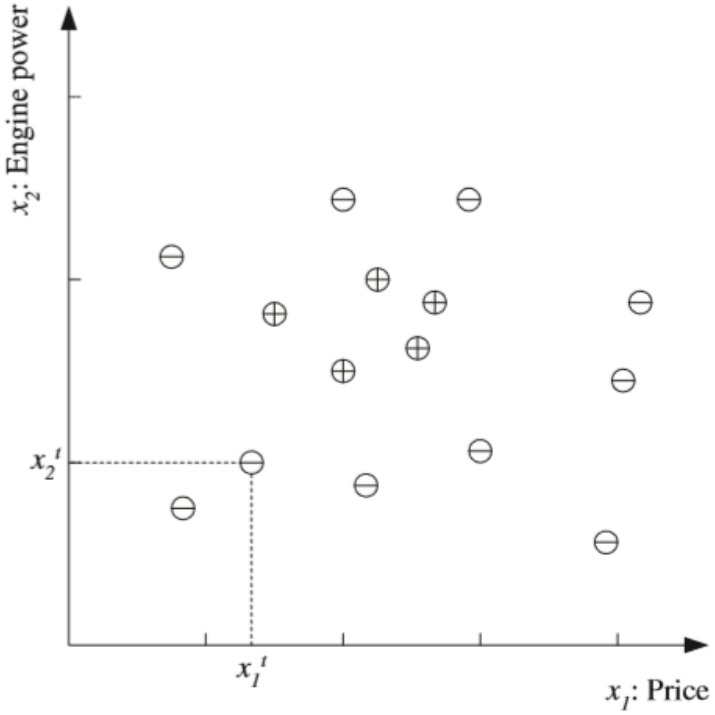

Matplotlib / Pandas / NumPy are the tools here!

# Core question




<u>Core Question</u> we are trying to answer (using training data):

What is the relationship $f$ between (*price*, *power*) and the Class $C$?




$$f: X \rightarrow Y$$

$$f: (price, power) \rightarrow Car\ class$$

$$f: (x_1, x_2) \subseteq \mathbb{R}^2 \rightarrow \{0,1\}$$

$$f: features \rightarrow categories$$

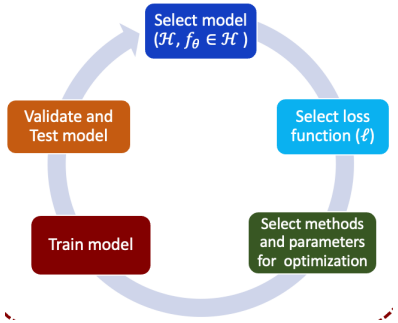
$$f: predictors \rightarrow predictions$$

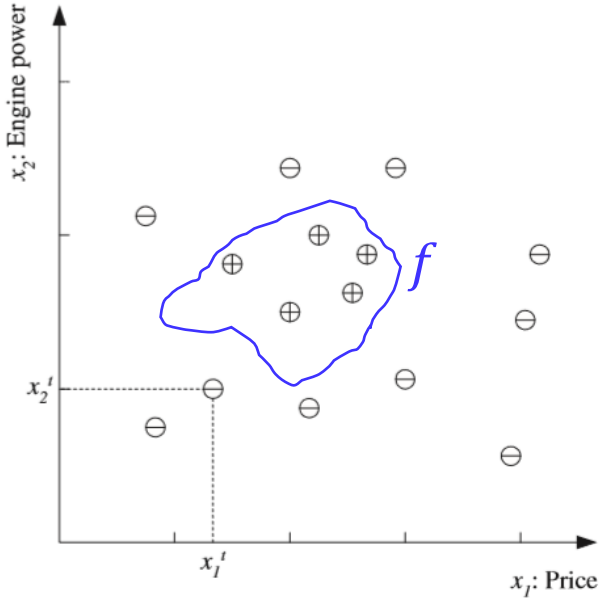
# Hypothesis about model properties

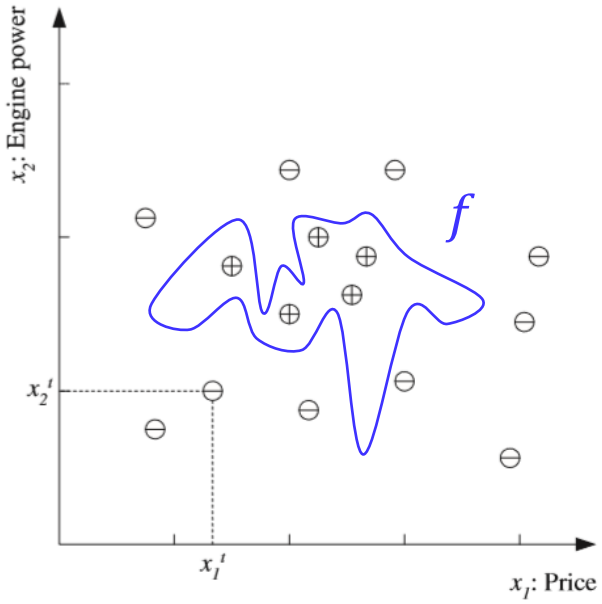
<u>Core Question</u>: What is the relationship $f$ between (*price, power*) and the Class $C$?

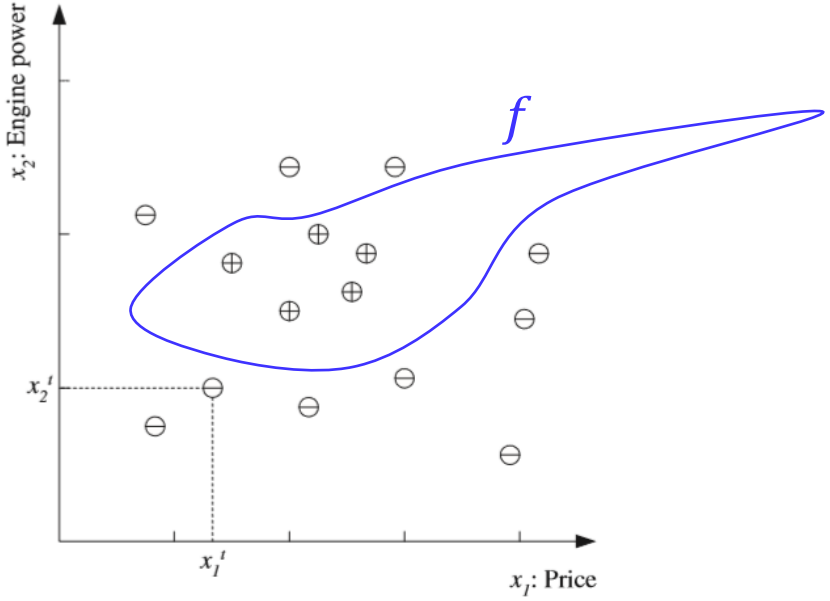
$f: (price, power) \rightarrow Car\ class$

**Hypothesis** about the ***form*** of (unknown) $f$:
to which *class of functions* $\mathcal{H}$, the searched mapping $f$, belongs to, or can be well approximated by



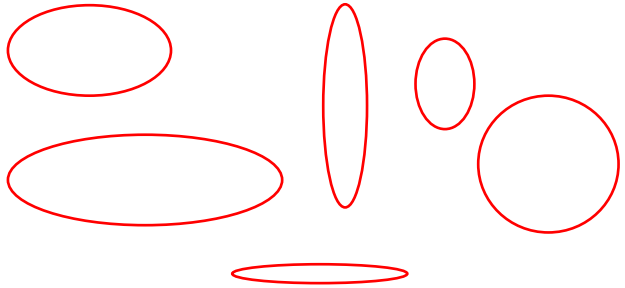



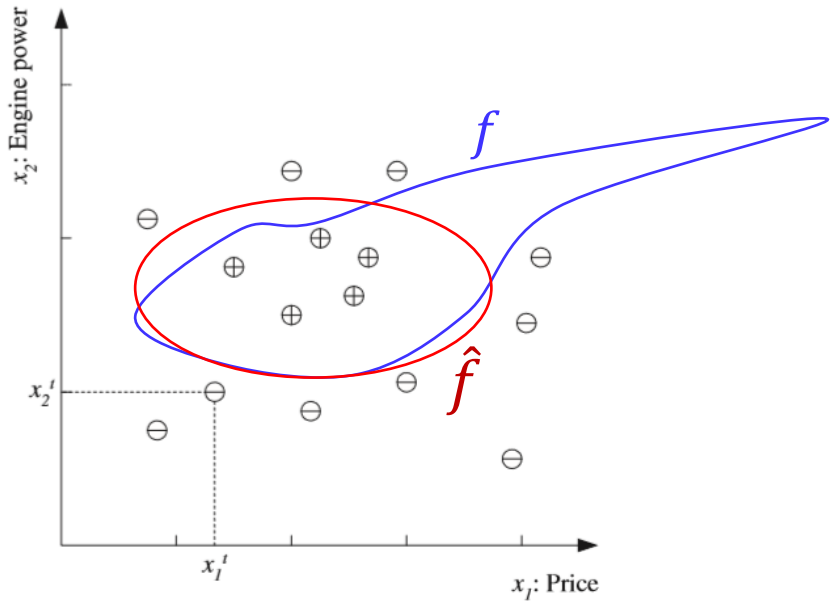
# Hypothesis about model properties

What we're looking for:

- Based on the training data , find in $\mathcal{H}$ a function $\hat{f} \approx f$ that <u>separates</u> the two classes in the feature space such that we can classify *future / unseen input data* with minimal errors

$\mathcal{H}$: {axis-aligned ellipses}






$\hat{f}$ = a good approximation of the **true** but <u>unknown</u> function $f$

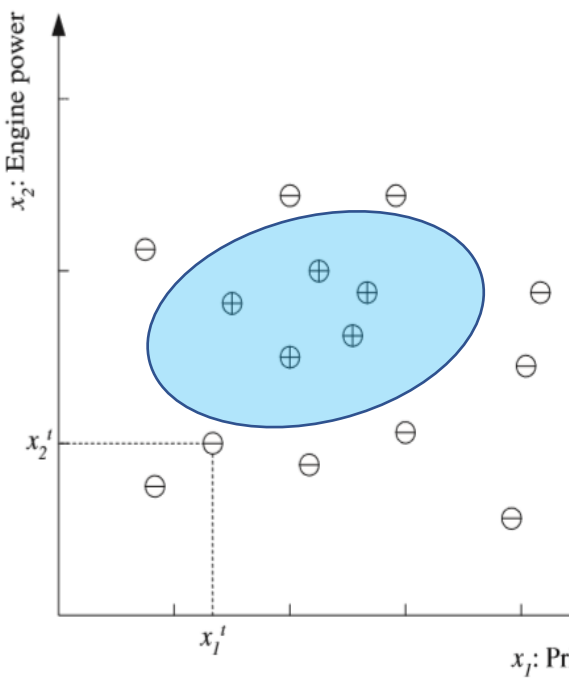
# Inductive bias

**Hypothesis** about the ***general form*** of $f$:
which *class of functions* $\mathcal{H}$ shall be selected to search for a mapping $\hat{f}$
that well approximates the true (unknown) mapping $f$?

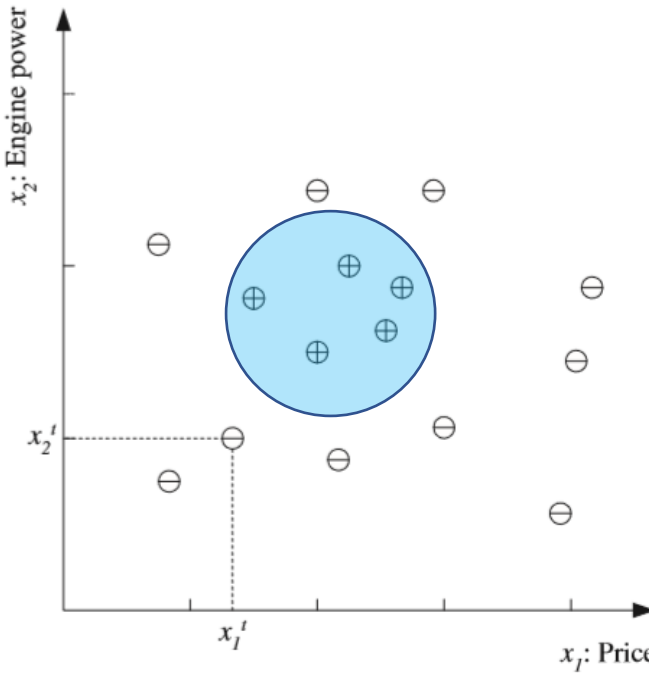
$$\hat{f} \approx f \qquad \hat{f} \in \mathcal{H}$$

- We have to <u>make a choice</u> → **Inductive bias**
- We will *explain* the data according to the hypothesis class $\mathcal{H}$ that we choose (that sets a ***bias***)
- We will use the data to <u>find</u> in $\mathcal{H}$ the mapping $\hat{f}$ that <u>best</u> approximates $f$, i.e., the best hypothesis in $\mathcal{H}$ given the data

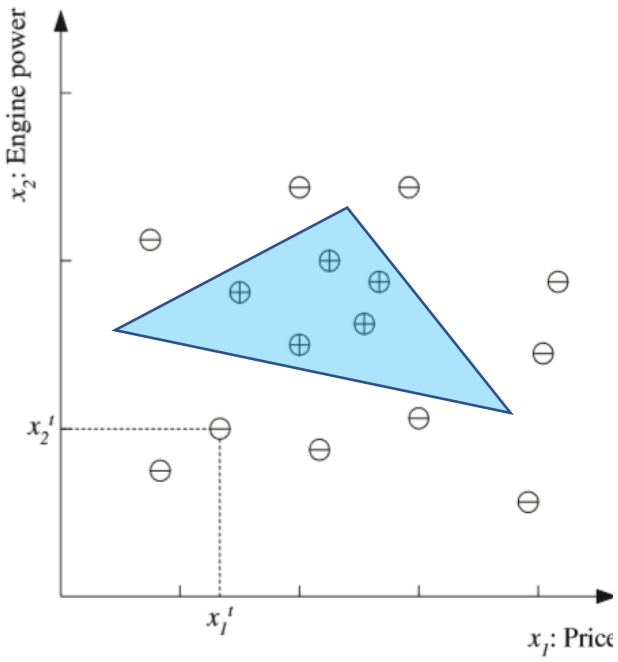
# Quiz time: which class of functions seems a good choice for $\mathcal{H}$?



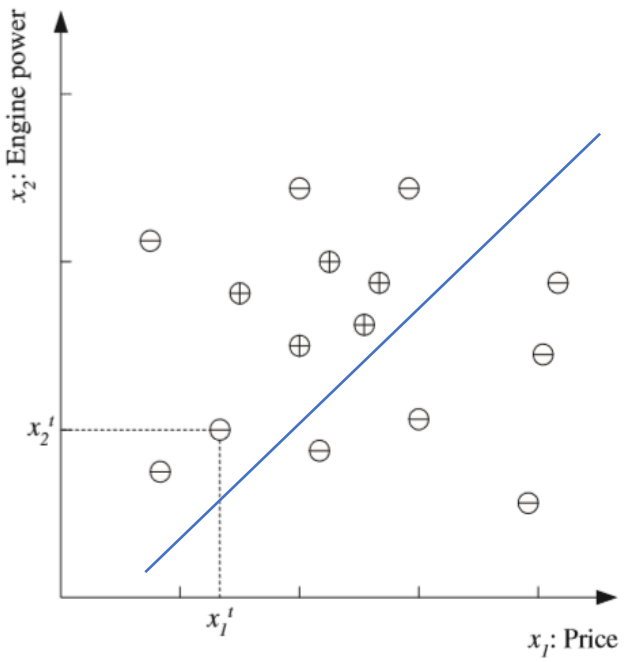

Do ellipses seem a good choice?




Circles?




Triangles




Lines?

- How many **parameters** do we have to learn for each one of these classes of hypotheses $\mathcal{H}$?

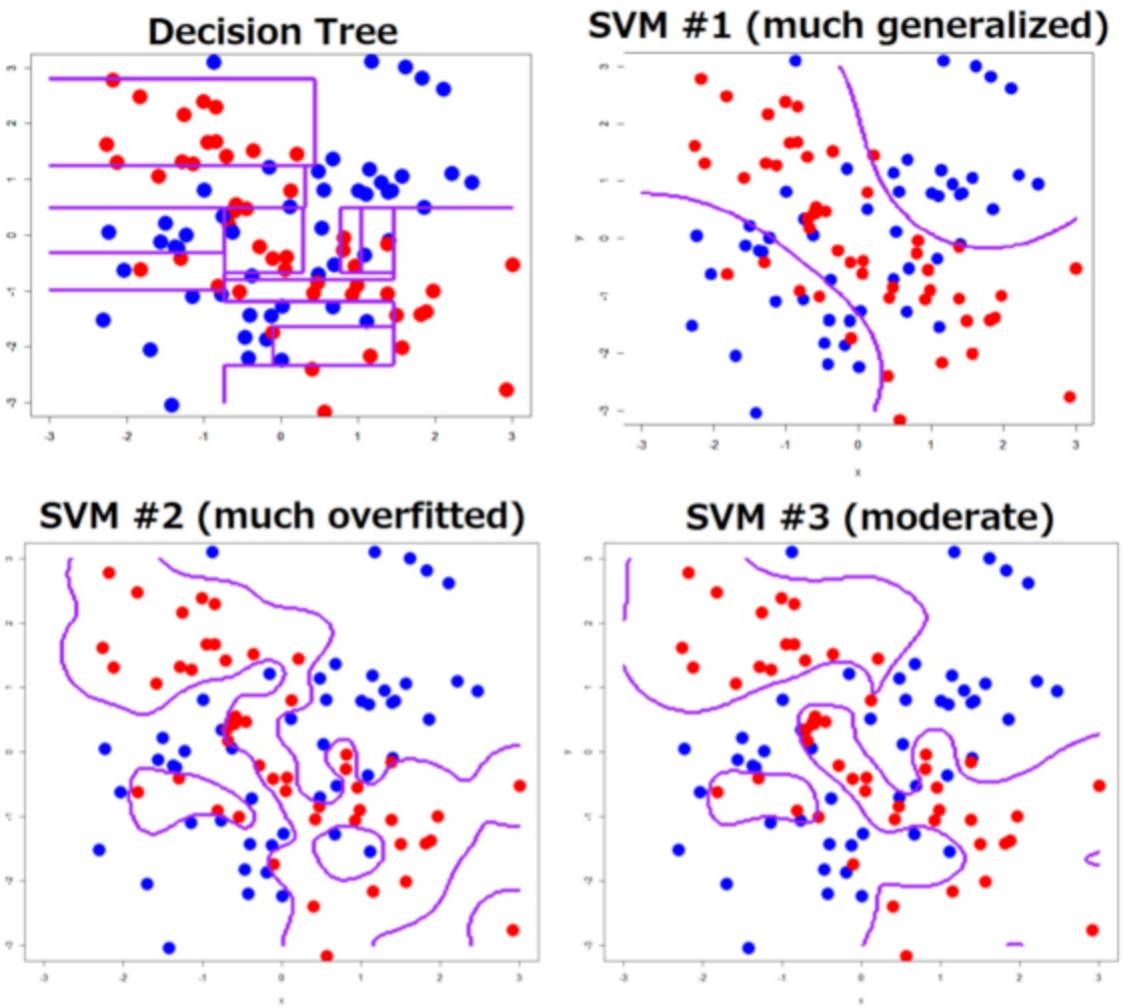
$f: (x_1, x_2) \rightarrow \{0,1\}$

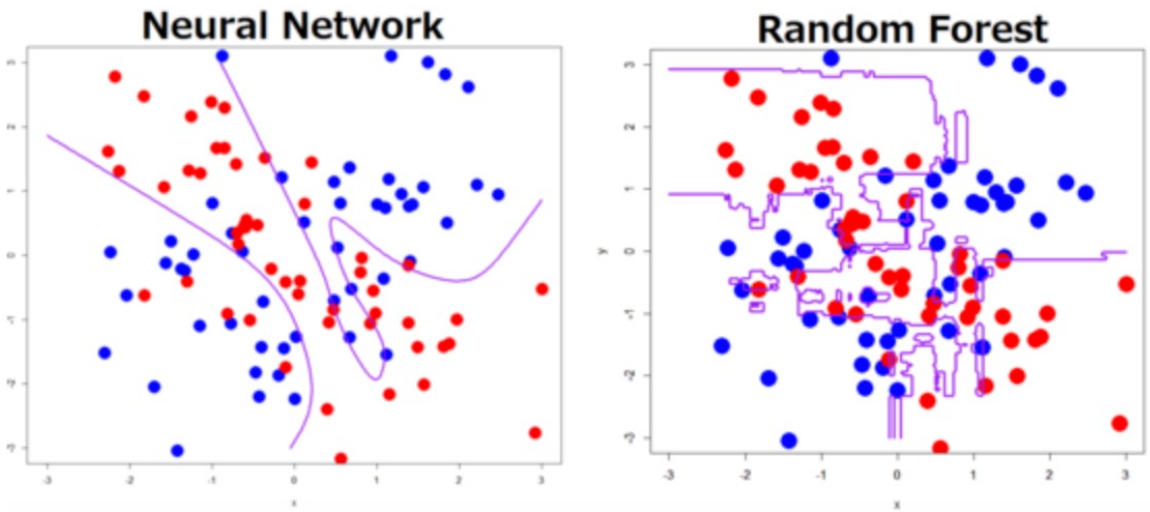
$$\hat{f}_{circle}(x_1, x_2) = (x_1 - x_{1c})^2 + (x_2 - x_{2c})^2 - r^2 = 0$$

$$\hat{f}_{ellipse}(x_1, x_2) = \frac{(x_1 \cos\alpha + x_2 \sin\alpha)^2}{a^2} + \frac{(x_1 \sin\alpha - x_2 \cos\alpha)^2}{b^2} - 1 = 0 \quad \text{(centered in (0,0))}$$

# I'm confused, do we *need* to define analytical function spaces?

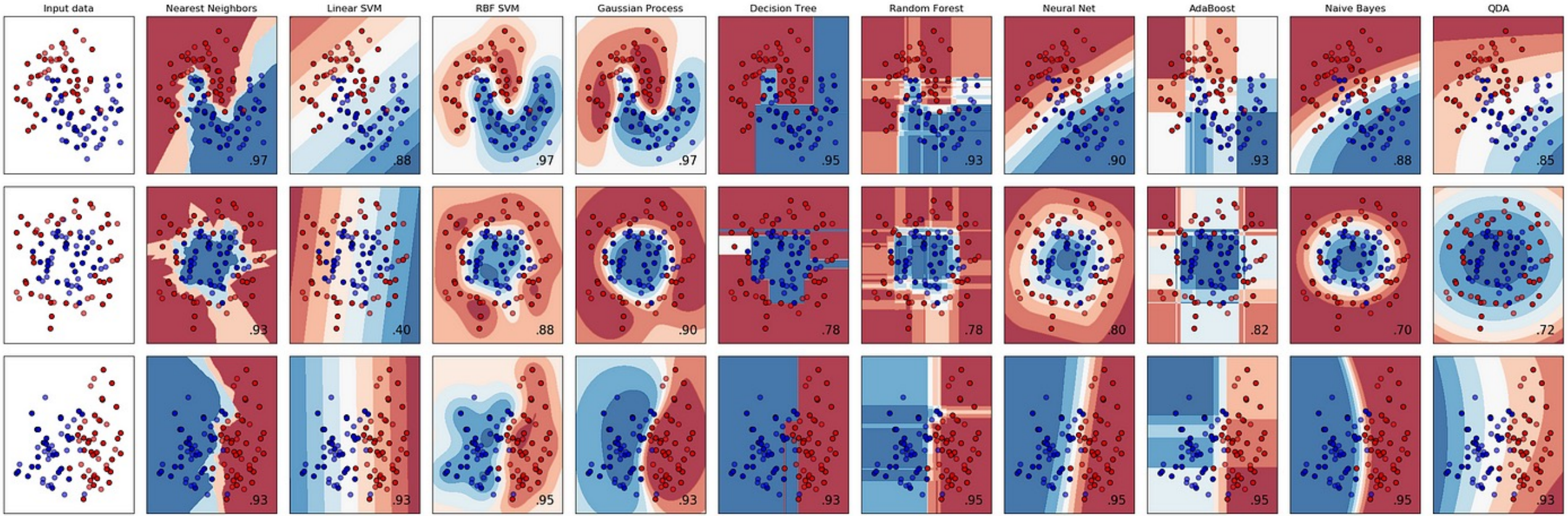
For a different binary classification problem (complex XOR), see the decision boundaries defined by different algorithms, i.e., by different hypothesis spaces, $\mathcal{H}$, **implicitly** defined by the algorithms and their parameters



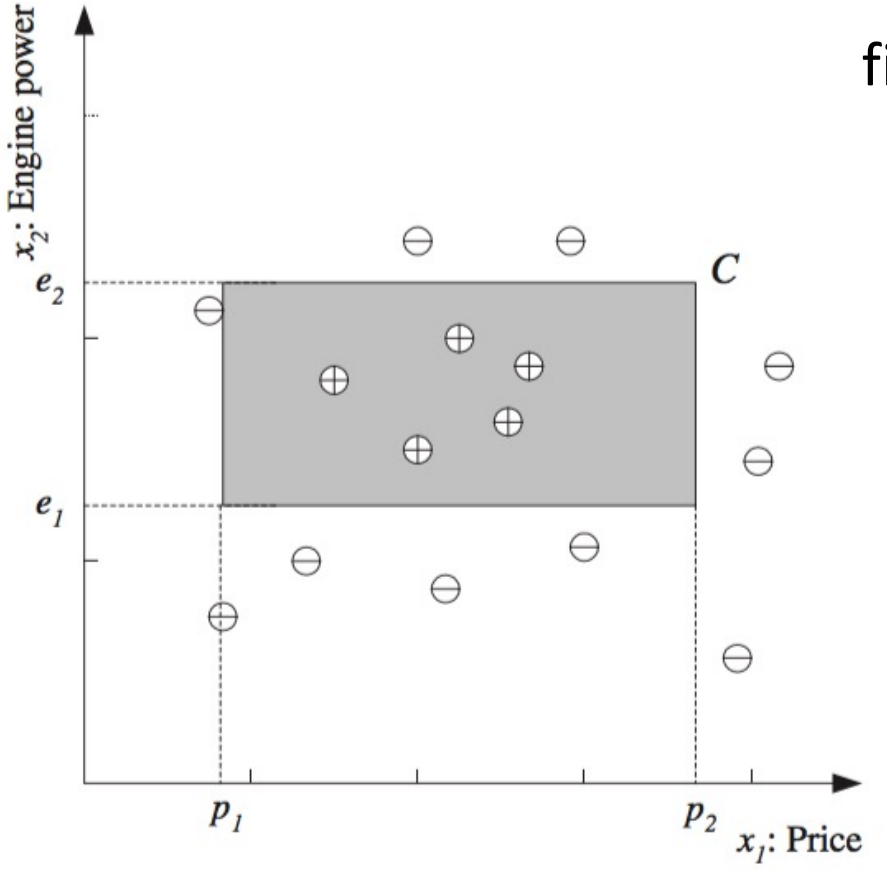



# I'm confused, do we *need* to define analytical function spaces?

For a different binary classification problem (complex XOR), see the decision boundaries defined by different algorithms, i.e., by different hypothesis spaces, $\mathcal{H}$, **implicitly** defined by the algorithms and their parameters



# Parametric hypothesis class: axis-aligned rectangles




Target of learning:

find one **particular hypothesis** $\hat{f} \in \mathcal{H}$ to approximate the (true) class $C$ *as closely (?) as possible* (i.e., the true $f$)

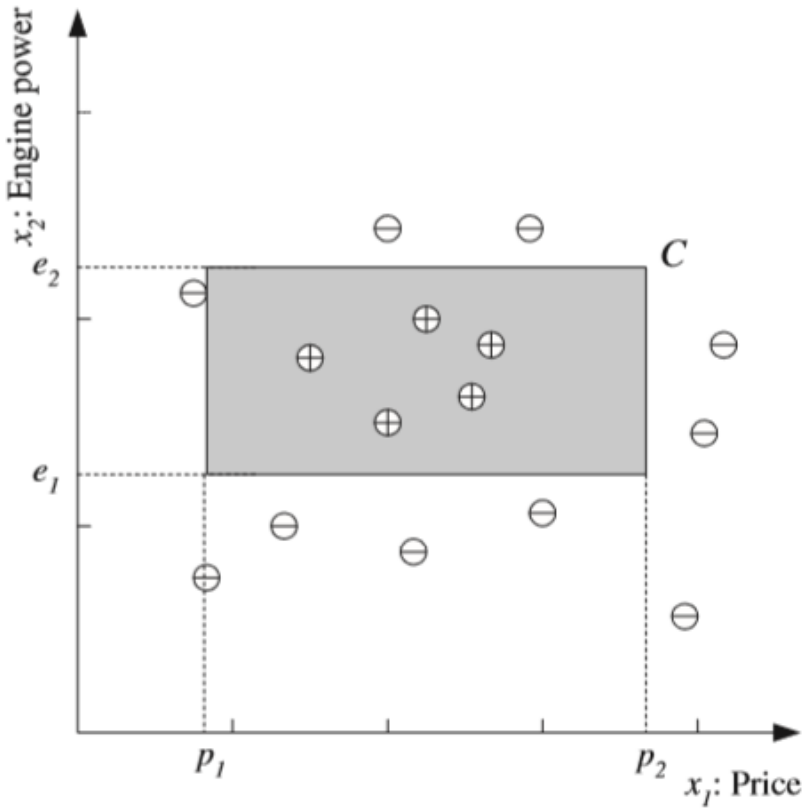
**Hypothesis class** from which we believe/assume $C$ is drawn: $\mathcal{H}$ = set of axis-aligned rectangles

$$(p_1 \leq \text{price} \leq p_2) \wedge (e_1 \leq \text{engine power} \leq e_2)$$

Learn a vector $\boldsymbol{\theta} = (p_1, p_2, e_1, e_2)$ of *four* parameters:

$$\hat{f}(\boldsymbol{x}) \triangleq f_\theta(\boldsymbol{x})$$

# How to choose the *best* specific hypothesis $\hat{f}$?




Let the shaded rectangle be the *true* class $C$, the true mapping $f$

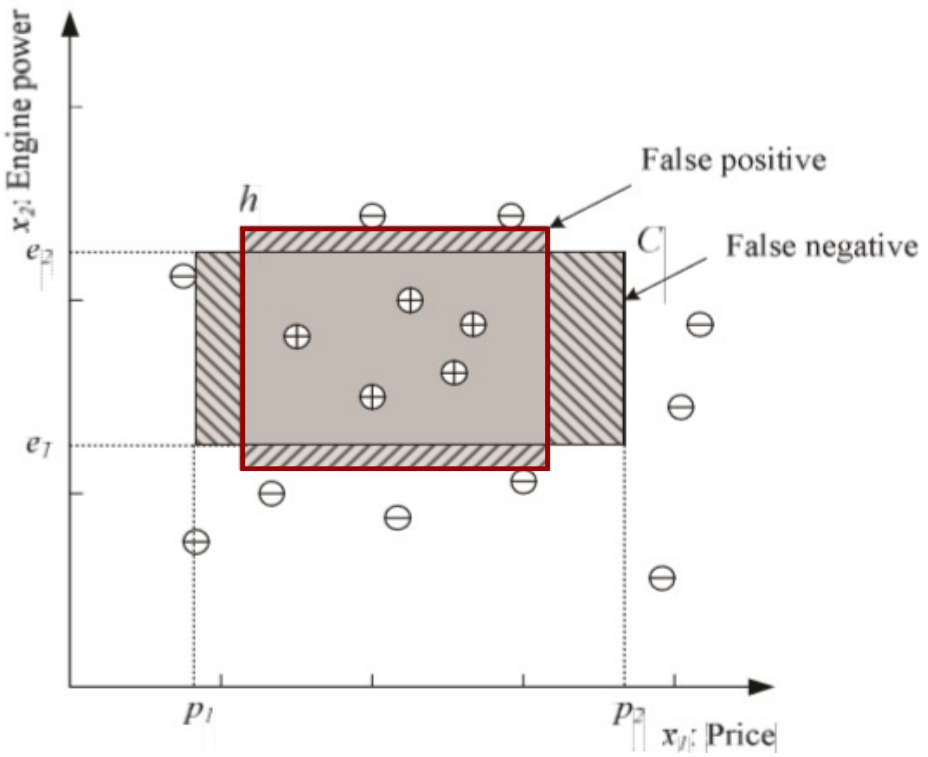
Problem:
We don't know $C$, we only have a set of examples drawn from $C$ …

# Consistent hypotheses

❖ One (reasonable?) choice is to look for a hypothesis $f_\theta$ that makes no mistakes <u>on available (training) data</u>

✓ **Consistent** *hypothesis:*
- Hypothesis that *fully agrees* with the data

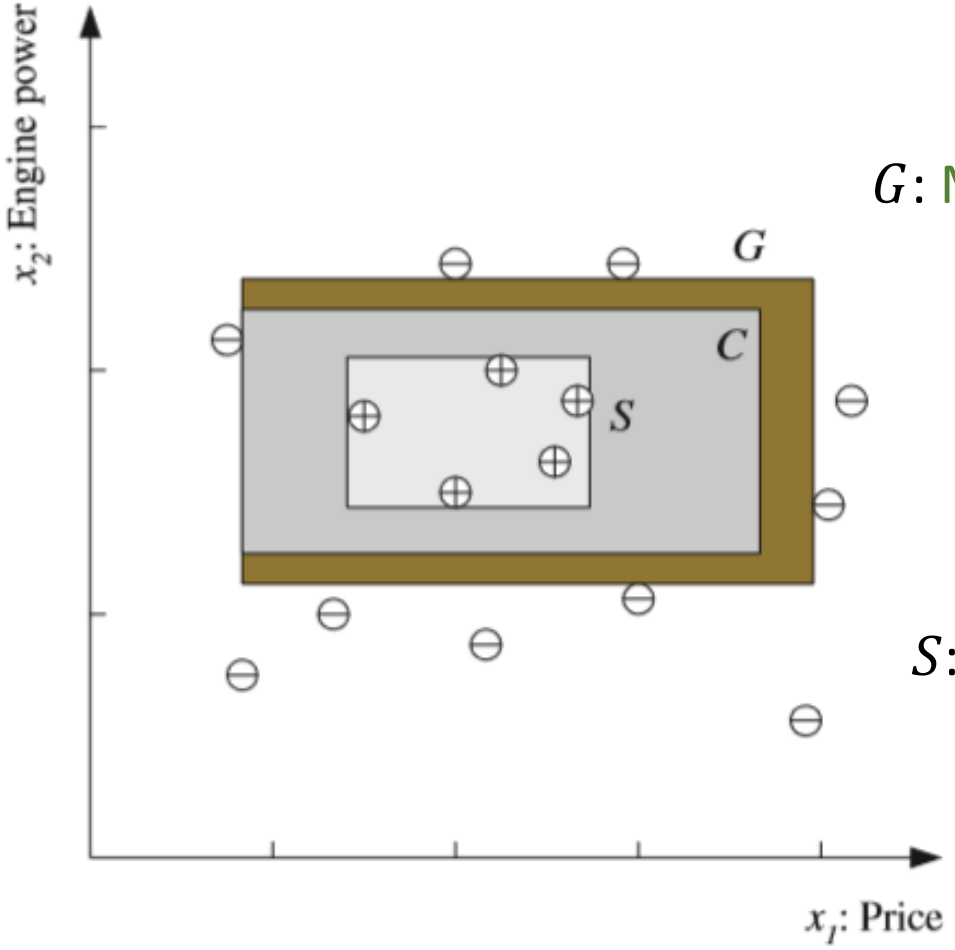
The red framed box: $f_\theta$ **fully *agrees*** with available data!




...but can still be wrong on **unseen** samples!
(based on the *true* class)

# Consistent hypotheses

- Different ways of being consistent



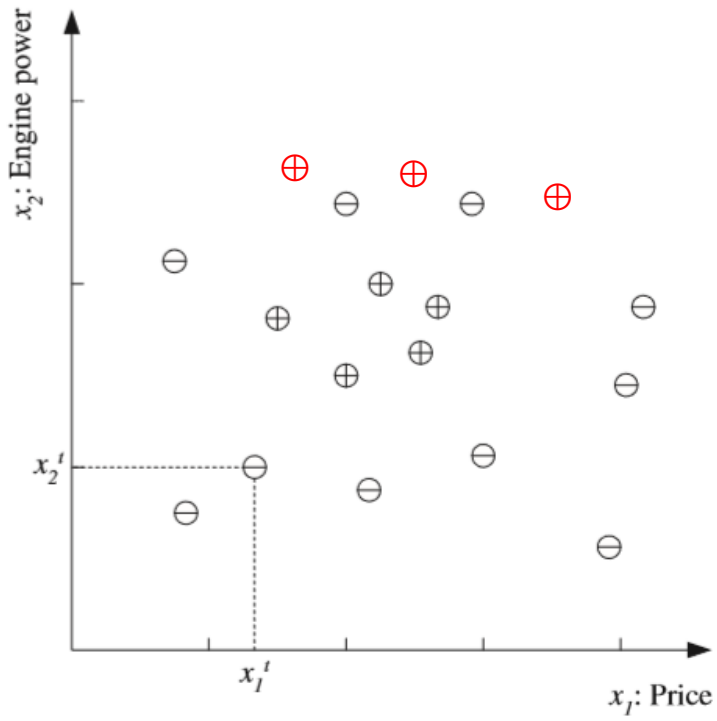

$G$: Most general consistent

$S$: Most specific consistent

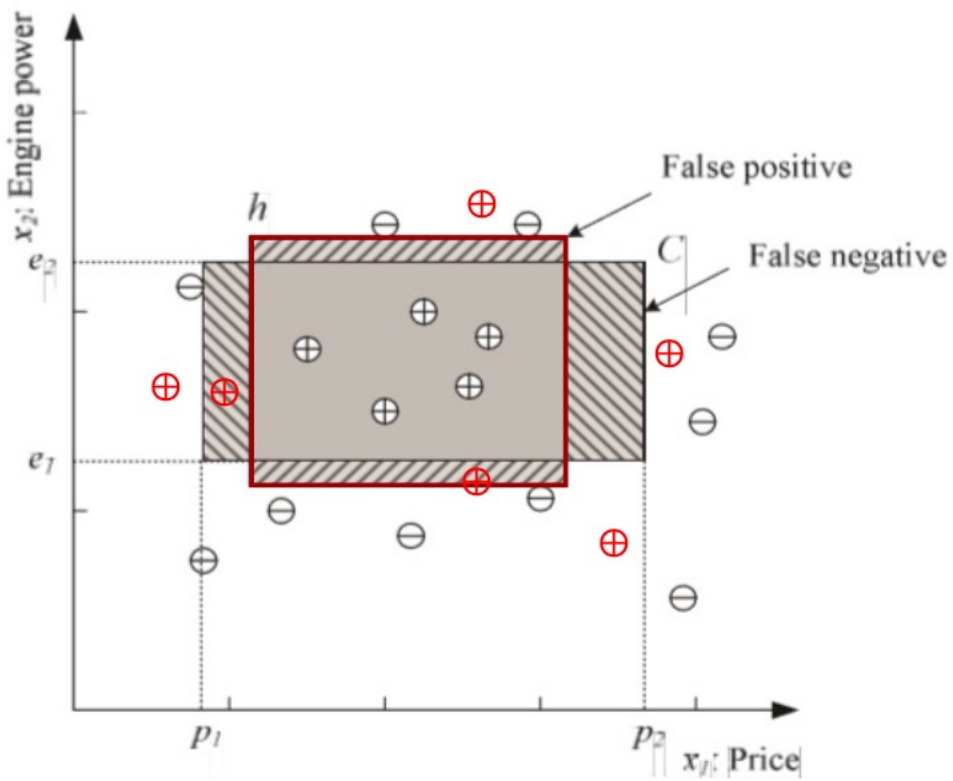
# Need to stick to consistency?

*Must* $f_\theta(\boldsymbol{x})$ <u>always</u> be consistent?

- Sometimes is it just impossible to be consistent <u>given the choice</u> of $\mathcal{H}$




- In general, being consistent with the available data, doesn't ensure zero/low errors on unseen data!
- Especially true for **small datasets**

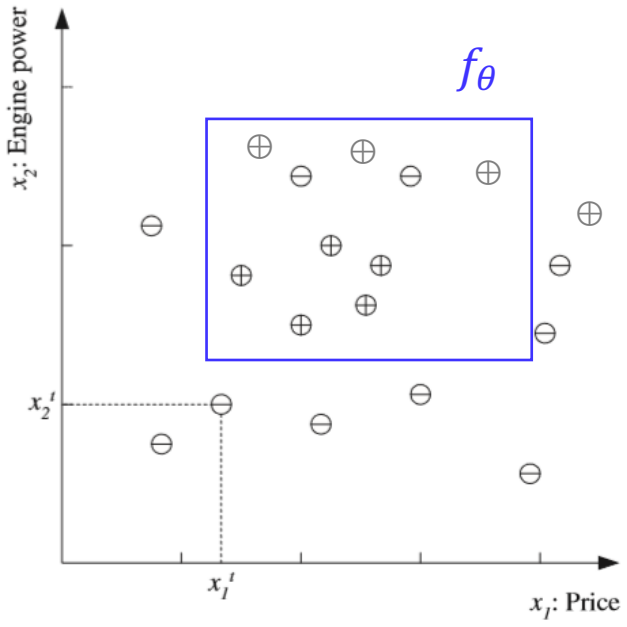


# How to choose the hypothesis (if consistency is not a must)?

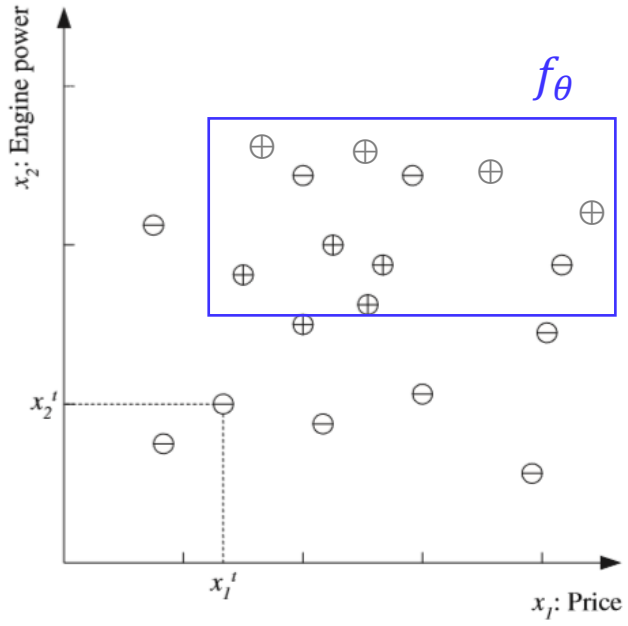
How do we evaluate *how* ***good*** a learned $f_\theta(\boldsymbol{x})$ is?

- ✓ For the training data in $\mathcal{D}$, we do have the labels, the target values
  - ➢ We can measure the **loss**, $\ell$, the amount of **error** in $\mathcal{D}$ with respect to the ground truth

**Loss function** $\ell$ for the car problem?




$loss = 3$

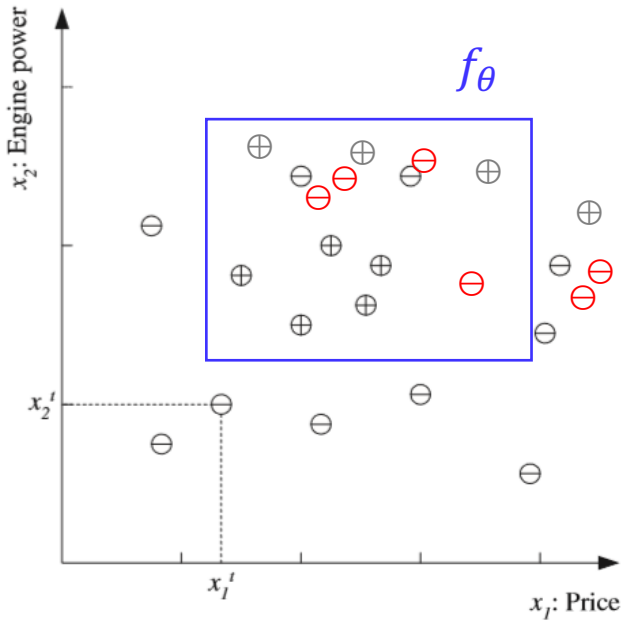



$loss = 4$

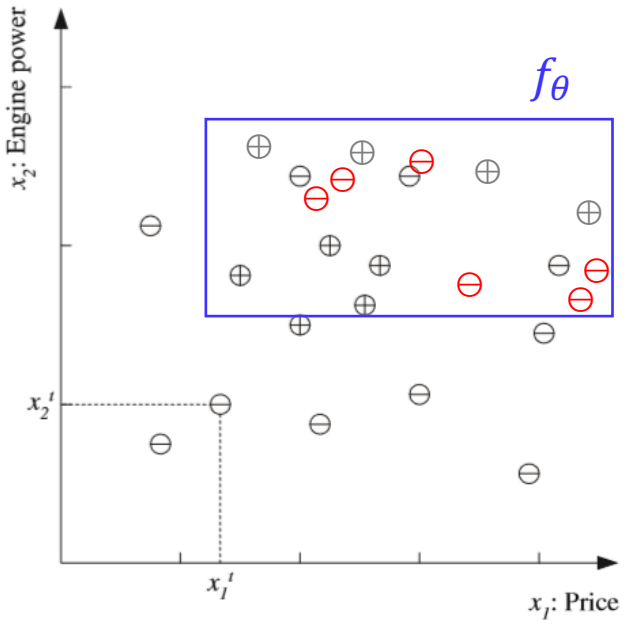
# How to choose the hypothesis (if consistency is not a must)?

How do we evaluate *how* ***good*** a learned $f_\theta(\boldsymbol{x})$ is?

- The loss on the training dataset is a measure of the **risk** of making future mistakes when using $f_\theta(\boldsymbol{x})$

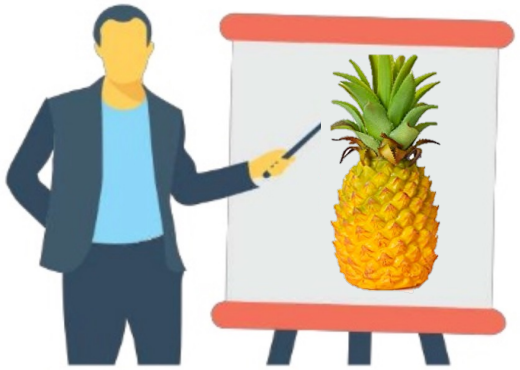


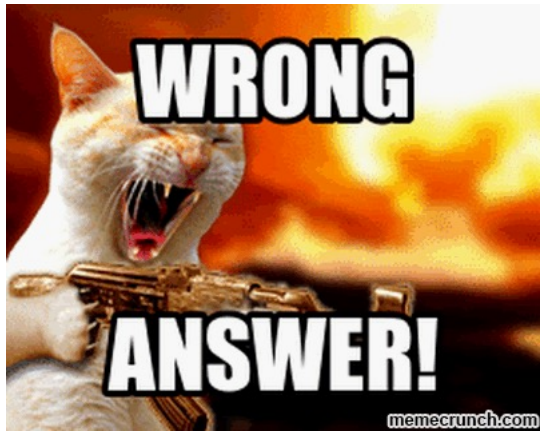

loss on new (unseen) data = 4



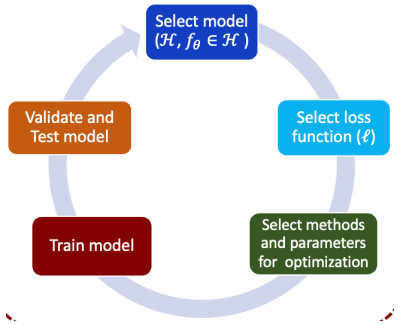

loss on new (unseen) data = 6

# Remember: Learning with a Teacher → Error correction






- ✓ **The model learned so far needs to be <u>modified</u>**
- ➢ *How?*
- ✓ **Accounting for the <u>error</u>**, quantified by the supervisor through the information carried by the label




Quantification of the error:
**Loss function!**

- Entity of correction should be somehow proportional to the magnitude of error, the loss in prediction
- We want to keep correcting / adjusting the parameters of the model if it makes mistakes

# Loss function: a core design choice

❖ **Loss function**, $\ell$**:** quantifies the <u>prediction/classification *error*</u> done by <u>our</u> hypothesis function $f_\theta(\boldsymbol{x})$ on **one training example** (→ Sum over the total training set)

$\ell$(*predicted value*, *true value*)

$$\boxed{\ell\big(f_\theta(\boldsymbol{x}),\ \ y(\boldsymbol{x})\big) \to \mathbb{R}}$$

→ Numeric quantification of the <u>closeness</u> between expected/true and predicted for the given example

- A loss function for the car problem?

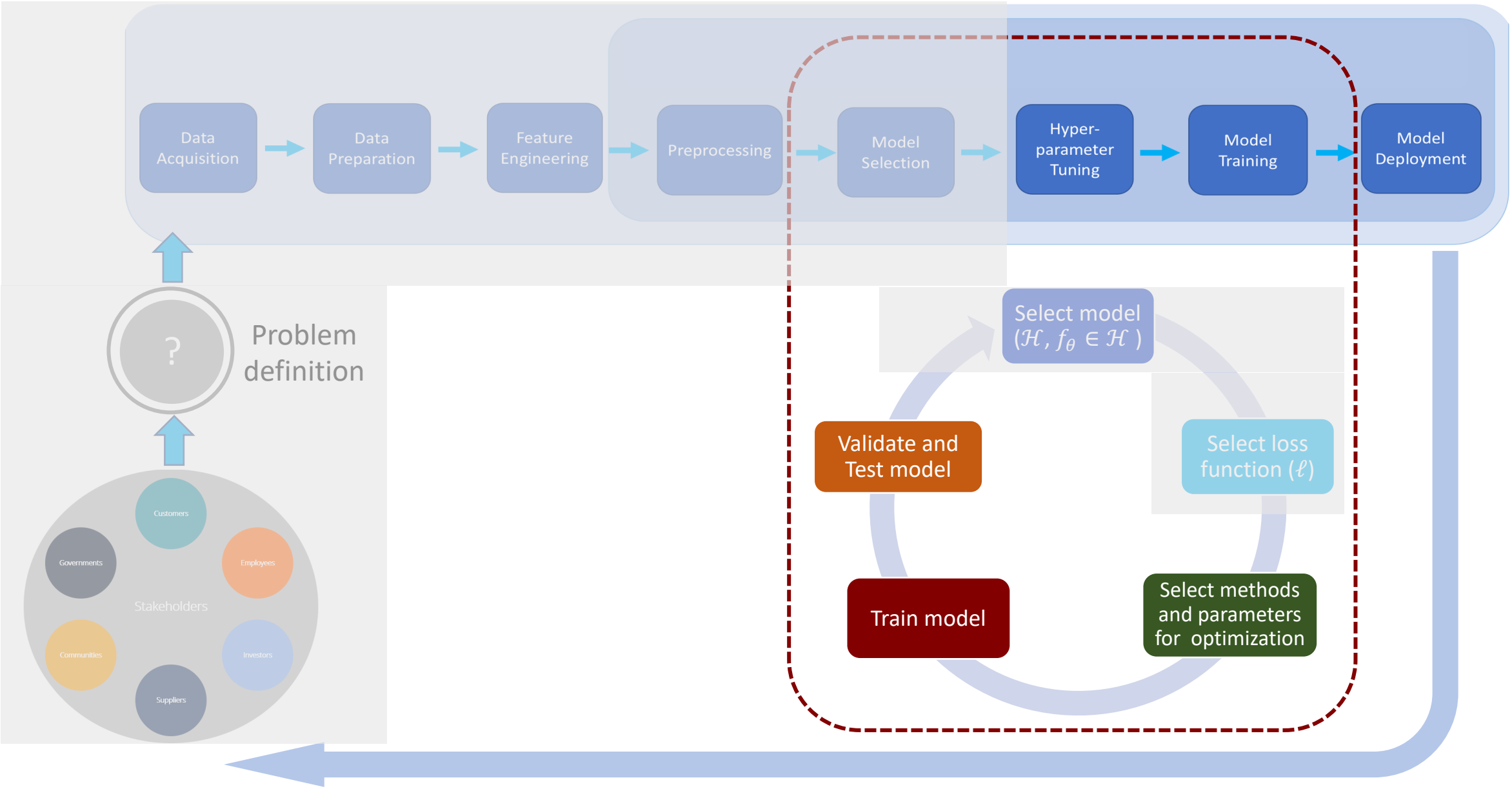
0/1 Loss

$$\ell\big(f_\theta(\boldsymbol{x}^{(k)}), y^{(k)}\big) = \begin{cases} 1 & \text{if } f_\theta\big(\boldsymbol{x}^{(k)}\big) \neq y^{(k)} \\ 0 & \text{if } f_\theta\big(\boldsymbol{x}^{(k)}\big) = y^{(k)} \end{cases}$$

$$\sum_{\forall\, \boldsymbol{x} \in D} \ell\big(f_\theta(\boldsymbol{x}), y(\boldsymbol{x})\big)$$

Count total errors on dataset

# …. still, need to optimize and train, and validate and test



# Train the model to minimize empirical error

**Train & Optimize the model**: Use the data in $D$ to (*iteratively*, if necessary) set the values of the parameter vector $\boldsymbol{\theta}$ such that the **loss** generated by $f_\theta$ on the data available in $D$ is **minimized**

**Optimization problem:**

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right) \quad \leftrightarrow \quad \boxed{\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)}$$

$$L_{emp}(\boldsymbol{\theta}; D) = \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

***Empirical* (mean) prediction error,**
**Empirical risk associated to $f_\theta$**
(sample average of losses, sample error)

$$\boxed{\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L_{emp}(\boldsymbol{\theta}; D)}$$

… Is this what are we *really* looking for?
🤔

# The Canonical Supervised ML Problem

1. Given a **dataset of training examples**, $D$, consisting of pairs of input features and target outputs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\},\ i = 1, \ldots, m$, and selected:
   - a **hypothesis function** $f_\theta$,
   - a **loss function** $\ell$,

   find (or approximate) **parameter values** $\boldsymbol{\theta}^*$ that **minimize the empirical error on** $D$:

$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

To be continued …